

FPGA Programming – VHDL Part

VHDL by Examples – Adder

Andrea Triossi – University of Padova – INFN Padova

Adder

- Elementary cell for adding two bits (binary digit)
- Extension to two n-bits numbers

A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
1	0	0	1	0
0	1	0	1	0
1	1	0	0	1
0	0	1	1	0
1	0	1	0	1
0	1	1	0	1
1	1	1	1	1

$$S_i = (A \bar{B} \bar{C} + \bar{A} B \bar{C} + \bar{A} \bar{B} C + A B C)_i$$

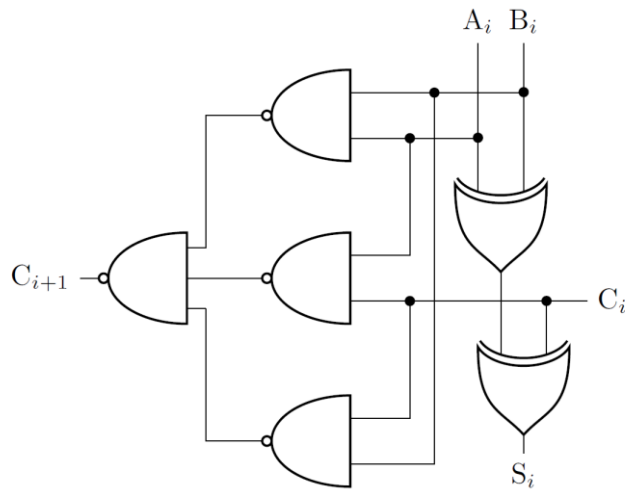
$$C_{i+1} = (A B \bar{C} + A \bar{B} C + \bar{A} B C + A B C)_i$$

$$S_i = (A \bar{B} + \bar{A} B)_i \bar{C}_i + (\bar{A} \bar{B} + A B)_i C_i = (A \oplus B)_i \oplus C_i$$

$$C_{i+1} = (A B + A C + B C)_i$$

Adder

- Elementary cell for adding two bits (binary digit)
- Extension to two n-bits numbers



$$S_i = (A \bar{B} \bar{C} + \bar{A} B \bar{C} + \bar{A} \bar{B} C + A B C)_i$$

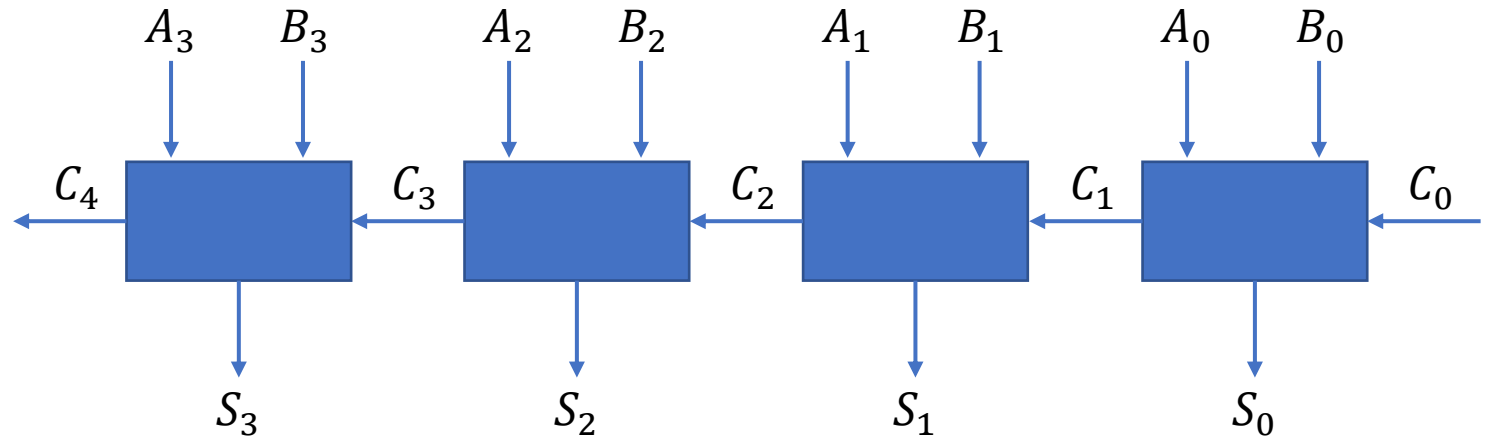
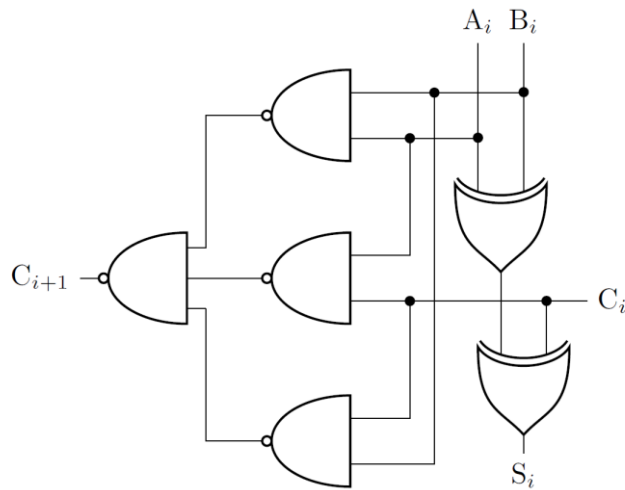
$$C_{i+1} = (A B \bar{C} + A \bar{B} C + \bar{A} B C + A B C)_i$$

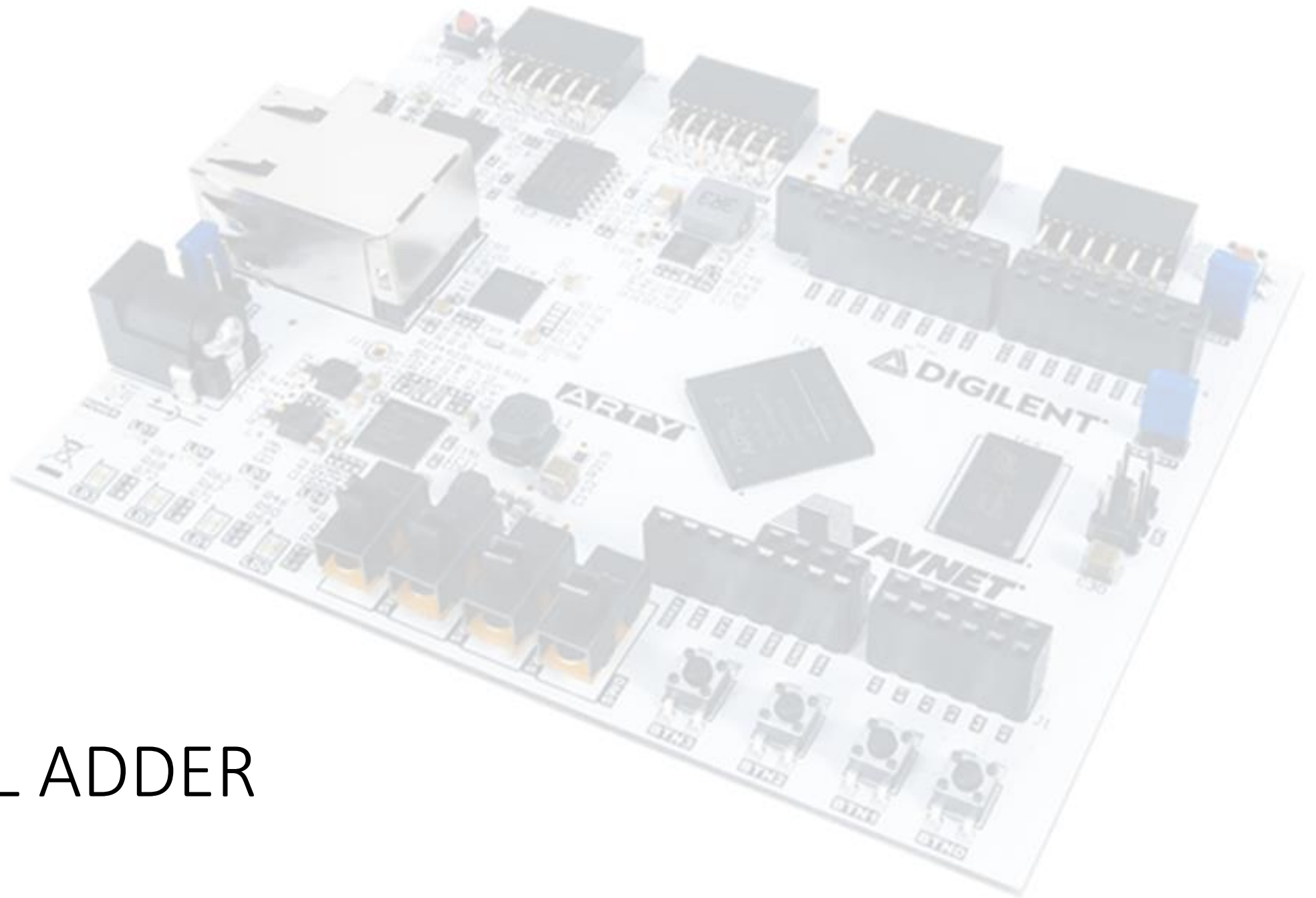
$$S_i = (A \bar{B} + \bar{A} B)_i \bar{C}_i + (\bar{A} \bar{B} + A B)_i C_i = (A \oplus B)_i \oplus C_i$$

$$C_{i+1} = (A B + A C + B C)_i$$

Adder

- Elementary cell for adding two bits (binary digit)
- Extension to two n-bits numbers





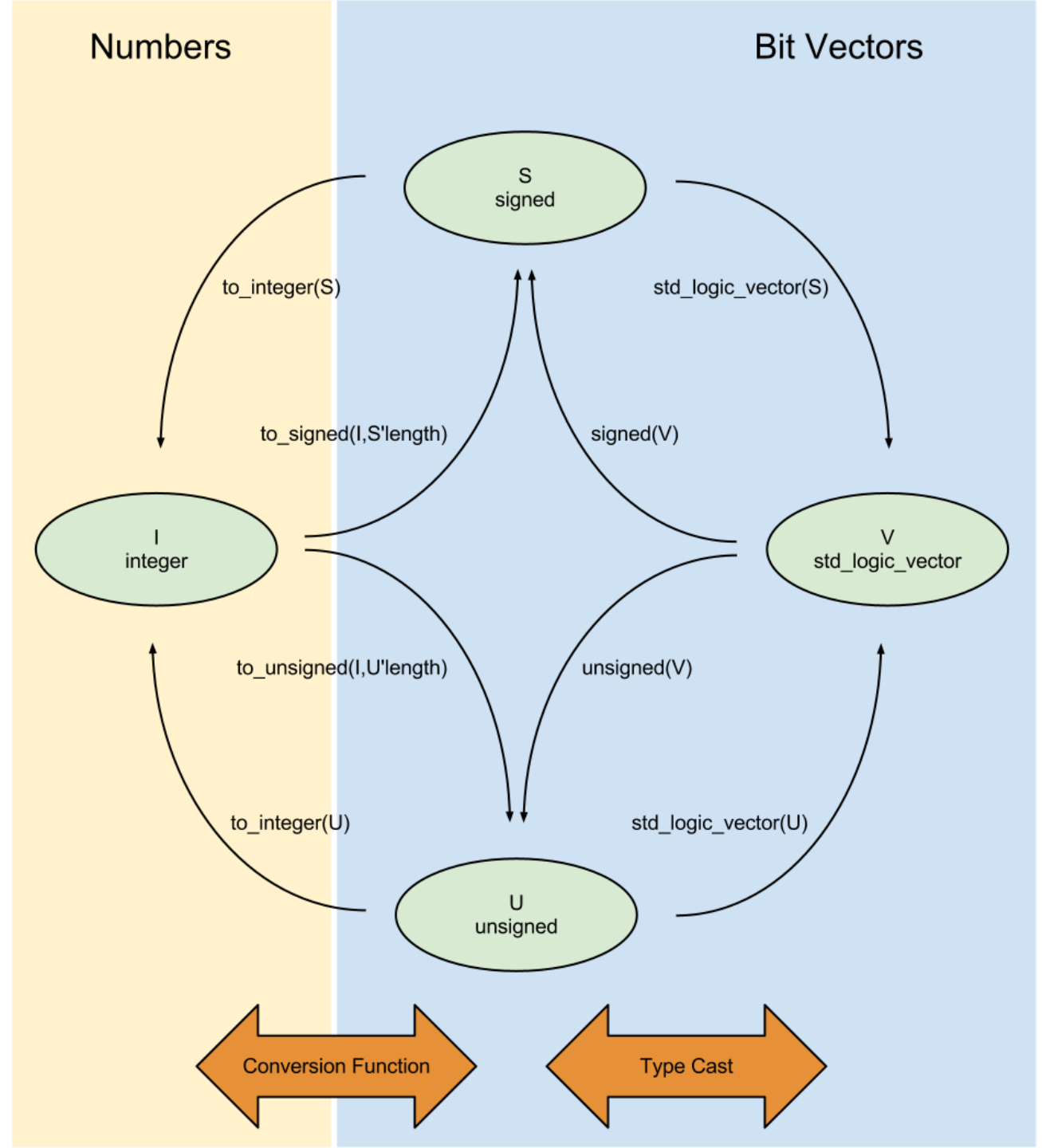
LAB RTL ADDER

2's complement operations

- Given k bit, we can represent the numbers in the interval $[-2^{k-1}, 2^{k-1} - 1] \subset \mathbb{Z}$
- The most significant bit tells us if the number is positive or negative
- For instance, 8 bit $\rightarrow$ 256 numbers from -128 to 127
 - $-2+5 \rightarrow$ 11111110+
00000101=
100000011
 - $5-3 \rightarrow$ 00000101+
11111100=
100000010

10000000	-128
...	...
11111110	-2
11111111	-1
00000000	0
00000001	1
00000010	2
...	...
01111111	127

Type conversions



Adder

- Adder in VHDL (with unsigned arithmetic)

```
entity NBitAdder is  
  generic (n: NATURAL :=4);  
  port (A, B: in std_logic_vector(n-1 downto 0);  
        Cin : in std_logic;  
        Sum : out std_logic_vector(n-1 downto 0);  
        Cout: out std_logic);  
end entity NBitAdder;
```

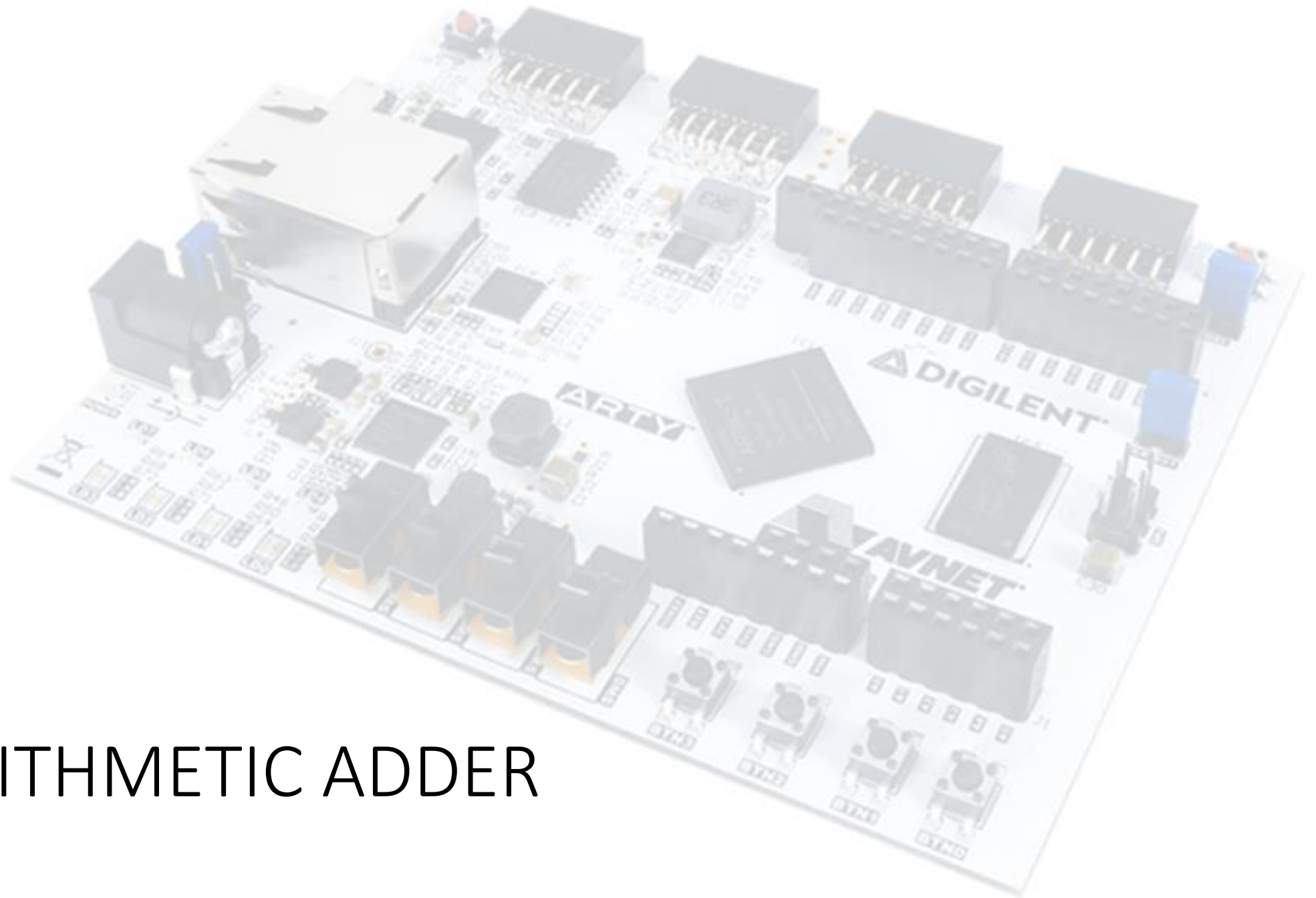
```
architecture unsigned of NBitAdder is  
  signal result : unsigned(n downto 0);  
  signal carry : unsigned(n downto 0);  
  constant zeros : unsigned(n-1 downto 0) := (others => '0');  
begin  
  carry <= (zeros & Cin);  
  result <= ('0' & unsigned(A)) + ('0' & unsigned(B)) + carry;  
  Sum <= std_logic_vector(result(n-1 downto 0));  
  Cout <= result(n);  
end architecture unsigned;
```


Adder

- Adder in VHDL (with signed arithmetic)

```
entity NBitAdder is  
  generic (n: NATURAL :=4);  
  port (A, B: in std_logic_vector(n-1 downto 0);  
        Cin : in std_logic;  
        Sum : out std_logic_vector(n-1 downto 0);  
        Cout: out std_logic);  
end entity NBitAdder;
```

```
architecture signed of NBitAdder is  
  signal result : signed(n downto 0);  
  signal carry : signed(n downto 0);  
  constant zeros : signed(n-1 downto 0) := (others => '0');  
begin  
  carry <= (zeros & Cin);  
  result <= (A(n-1) & signed(A)) + (B(n-1) & signed(B)) + carry;  
  Sum <= std_logic_vector(result(n-1 downto 0));  
  Cout <= result(n);  
end architecture signed;
```



LAB ARITHMETIC ADDER